

Relatório Parcial — Esteira Separadora (Firmware v3.0)

Sistemas Operacionais Embarcados (SOE) — 2026.1 Autores: Paulo Caleb Fernandes da Silva, Felipe de Castro

1. Descrição do Hardware Controlado

O sistema é composto por uma arquitetura híbrida que divide as responsabilidades entre um microprocessador de alto nível e um microcontrolador dedicado ao controle direto dos dispositivos físicos da esteira.

O **Raspberry Pi 3B** atua como o processador de alto nível, sendo responsável por executar os algoritmos de visão computacional para leitura de QR codes e classificação dos objetos transportados. Uma webcam USB acoplada ao Raspberry Pi captura as imagens no momento adequado do ciclo de operação. As decisões de roteamento são transmitidas ao microcontrolador via comunicação serial UART.

O **STM32G070** é o microcontrolador responsável pelo controle direto dos atuadores e pela leitura dos sensores. Opera a uma frequência de CPU de 64 MHz, obtida por meio do PLL interno a partir do oscilador HSI de 16 MHz (PLLN = 8, PLLM = 1, PLLR = 2). Os periféricos utilizados estão detalhados a seguir.

Timer	Função
TIM15	PWM fixo de 200 Hz para modulação dos lasers
TIM3	Input Capture + DMA para medição de frequência dos 4 sensores
TIM16	PWM de duty variável (50 Hz) para controle do servo motor
TIM6	Base de tempo de 1 µs para controle do motor de passo via interrupção

A **USART2** é configurada a 115200 bps, 8N1, sem controle de fluxo por hardware, para a comunicação serial com o Raspberry Pi 3B.

2. Sensores Laser — Detecção Síncrona por Portadora

Os quatro sensores laser operam sobre o princípio da **detecção síncrona por ausência de portadora**. O TIM15 gera um sinal PWM de 200 Hz com duty cycle de 50% (CCR = 25, ARR = 49, prescaler = 6399 sobre 64 MHz), que modula o acionamento dos transmissores laser. O receptor de cada sensor entrega ao STM32 um sinal digital pulsado nessa mesma frequência enquanto o feixe estiver livre.

A medição é feita pelo **TIM3 em modo Input Capture com DMA** (prescaler = 63, ARR = 65535, base de tempo = 1 µs/tick). Dois valores de captura consecutivos na borda de descida são armazenados em um vetor de dois elementos por DMA. A diferença entre as capturas fornece o período do sinal, e a frequência é calculada como:

$$SENSn_freq = 1.000.000 / SENSn_capture \quad [Hz]$$

Quando um objeto interrompe o feixe, a portadora desaparece e a diferença de captura torna-se zero. Nesse caso, `SENSn_flag` é setada em 1 (objeto detectado) e `SENSn_freq` é zerada. O período de amostragem é controlado por `sensUPDT = 10 ms`, que deve ser pelo menos o dobro do período do sinal da portadora (5 ms), garantindo duas bordas capturáveis por ciclo.

A frequência esperada de 200 Hz em todos os quatro sensores é verificada durante o procedimento de inicialização para confirmar que os feixes estão livres e os sensores operacionais antes que o handshake com o Raspberry Pi seja aceito.

3. Servo Motor

O servo motor ES08MA é controlado pelo **TIM16** em modo PWM (prescaler = 63, ARR = 19999, base de tempo = 1 µs/tick, período total = 20 ms = 50 Hz). O mapeamento de ângulo para largura de pulso é realizado pela função `SetServoAngle()`:

```
pulse_length = SERVO_MIN_US + (angle × (SERVO_MAX_US - SERVO_MIN_US) /  
SERVO_MAX_ANGLE)
```

onde `SERVO_MIN_US = 600 µs`, `SERVO_MAX_US = 2300 µs` e `SERVO_MAX_ANGLE = 180°`. O valor calculado é carregado diretamente no registrador CCR do TIM16.

Na aplicação, dois ângulos são utilizados: `closeAngle = 0°` (cancela fechada, rota A) e `openAngle = 45°` (cancela aberta, rota B).

4. Motor de Passo

O motor de passo é acionado por um driver **A4988** e controlado pelo **TIM6** (prescaler = 63, ARR variável, base de tempo = 1 µs/tick). A velocidade é ajustada pela função `stepperSetSpeed()`, que recalcula o ARR conforme:

```
TIM6_intFreq = 2 × stepsPerSeconds  
newARR = (TIM6_FREQUENCY / TIM6_intFreq) - 1
```

O fator 2 é necessário porque a ISR alterna o pino STEP a cada interrupção (toggle), de modo que um pulso completo (borda de subida + descida) corresponde a dois eventos de interrupção. A velocidade é limitada entre `STEPPER_MIN_VEL = 300 passos/s` e `STEPPER_MAX_VEL = 420 passos/s`.

A ISR `STEPPER_TIM6_ISR()`, chamada pelo callback `HAL_TIM_PeriodElapsedCallback()`, executa a lógica de contagem de passos:

1. Se `stepperEN == 1` e `targetSteps > 0`: alterna o pino STEP; quando `pulse_ctrl == 1` (borda de subida), decrementa `targetSteps`.
2. Se `targetSteps == 0`: força o pino STEP em nível baixo e seta `movement_done_flag = 1`.

A variável `pulse_ctrl` inverte a cada ISR, garantindo que o decremento ocorra somente uma vez por pulso completo.

Três modos de parada estão implementados:

- `stepperStopDisengaged()`: desliga o driver A4988 (pino SLEEP em nível baixo), libera o eixo.
- `stepperStopEngaged()`: mantém o driver energizado (pino SLEEP em nível alto), trava o eixo.
- `stepperFollowSteps()`: habilita o driver, reseta os controles e ativa o TIM6 para retomar a geração de pulsos.

Na FSM, o motor solicita continuamente `stepperMove(350, 0)` (350 passos para frente) nos estados que requerem movimento, garantindo que a esteira não pare enquanto o objeto estiver em trânsito.

5. Protocolo de Comunicação UART

A comunicação entre o STM32 e o Raspberry Pi 3B é realizada por quadros binários estruturados sobre a USART2 (115200 bps, 8N1). A recepção é feita em modo **interrupt-driven** (`HAL_UART_Receive_IT`, 1 byte por vez), processada por uma máquina de estados de três estágios no callback `HAL_UART_RxCpltCallback()`.

5.1 Estrutura dos Quadros

Quadro RX (Raspberry Pi → STM32):

Byte 0	Byte 1	Byte 2 (opcional)
<code>0xAA</code> START	CMD	DATA

O byte DATA só está presente quando `CMD = 0xE5` (`SET_STPR_TGT_STPS_MSG`), que carrega a quantidade de passos como argumento.

Quadro TX (STM32 → Raspberry Pi):

Tipo	Byte 0	Byte 1	Byte 2 (opcional)
Confirmação / Msg	<code>0x90</code>	payload/eco	—
Erro	<code>0x91</code>	—	—
Telemetria	<code>0x90</code>	<code>0x55</code>	STATUS_BYTE

5.2 Tabela de Comandos

Categoria	Byte	Identificador	Direção	Descrição
Handshake	<code>0x10</code>	<code>SYS_RDY_MSG</code>	RBPI3 → STM32	Sinaliza que o RBPI3 está pronto
Handshake	<code>0x01</code>	<code>SYS_INIT_MSG</code>	STM32 → RBPI3	Confirma inicialização do sistema
FSM RX	<code>0xDA</code>	<code>ROUTE_A_RECEIVE_MSG</code>	RBPI3 → STM32	Encaminha objeto para a Rota A

Categoria	Byte	Identificador	Direção	Descrição
FSM RX	0xDB	ROUTE_B_RECEIVE_MSG	RBPi3 → STM32	Encaminha objeto para a Rota B
FSM TX	0xA0	OBJ_DETECTED_MSG	STM32 → RBPi3	Objeto detectado no sensor 1
FSM TX	0xC0	CLSS_REQUEST_MSG	STM32 → RBPi3	Objeto sob a câmera, solicita classificação
FSM TX	0xFA	ROUTE_A_FWRDNG_MSG	STM32 → RBPi3	Confirmação de encaminhamento para Rota A
FSM TX	0xFB	ROUTE_B_FWRDNG_MSG	STM32 → RBPi3	Confirmação de encaminhamento para Rota B
FSM TX	0xBA	ROUTE_A_SCCSS_DLVRV_MSG	STM32 → RBPi3	Entrega confirmada na Rota A
FSM TX	0xBB	ROUTE_B_SCCSS_DLVRV_MSG	STM32 → RBPi3	Entrega confirmada na Rota B
Controle async	0xE1	LIGHT_EN_MSG	RBPi3 → STM32	Liga o flash
Controle async	0xD1	LIGHT_DISABLE_MSG	RBPi3 → STM32	Desliga o flash
Controle async	0xE2	GATE_OPEN_MSG	RBPi3 → STM32	Abre a cancela (servo)
Controle async	0xD2	GATE_CLOSE_MSG	RBPi3 → STM32	Fecha a cancela (servo)
Controle async	0xE3	STPR_EN_MSG	RBPi3 → STM32	Engaja o motor de passo
Controle async	0xD3	STPR_DISABLE_MSG	RBPi3 → STM32	Libera o motor de passo
Controle async	0xE4	SET_STPR_FORWARD_MSG	RBPi3 → STM32	Define direção: frente
Controle async	0xD4	SET_STPR_BACKWARD_MSG	RBPi3 → STM32	Define direção: trás
Controle async	0xE5	SET_STPR_TGT_STPS_MSG	RBPi3 → STM32	Define nº de passos (DATA = valor)
Modo	0xDD	DEBUG_MODE_TOGGLE_MSG	RBPi3 → STM32	Alterna entre modo FSM e Debug
Modo	0x11	MODE_FSM_MSG	STM32 → RBPi3	Informa modo FSM ativo

Categoria	Byte	Identificador	Direção	Descrição
Modo	0x22	MODE_DEBUG_MSG	STM32 → RBPi3	Informa modo Debug ativo
Controle	0x33	SW_RESET_MSG	RBPi3 → STM32	Solicita reset por software (NVIC)
Telemetria	0x55	SENS_STATUS_MSG	STM32 → RBPi3	Envia STATUS_BYTE dos 4 sensores
Resposta	0x90	CMD_OK_MSG	STM32 → RBPi3	Confirmação positiva
Resposta	0x91	CMD_ERR_MSG	STM32 → RBPi3	Erro de reconhecimento

5.3 Parser RX — Máquina de Estados

A recepção de quadros é gerenciada por uma FSM de três estados na ISR de recepção UART:

```

RX_WAIT_START → byte == 0xAA → RX_WAIT_CMD
RX_WAIT_CMD → CMD != 0xE5 → frame completo (2 bytes) → RX_WAIT_START
RX_WAIT_CMD → CMD == 0xE5 → RX_WAIT_DATA
RX_WAIT_DATA → → frame completo (3 bytes) → RX_WAIT_START

```

Qualquer byte fora de frame recebido no estado **RX_WAIT_START** é silenciosamente descartado, protegendo o parser contra ruído ou dados espúrios no barramento.

O frame completo é sinalizado pela flag **rxFrameReady**, com o comando em **rxCMD** e o dado opcional em **rxDATA**.

5.4 Transmissão TX

As funções de transmissão são:

- **sendFrame(status, payload)**: envia 2 bytes **[status][payload]**. Se **status == 0x91** (CMD_ERR), envia apenas 1 byte.
- **telemetryFrame(status, cmd, data)**: envia 3 bytes **[0x90][0x55][STATUS_BYTE]**.

As mensagens espontâneas da FSM são enviadas por flags no loop principal (ex.: **objDetectedMSG_flag**, **classificationRequestMSG_flag**), evitando chamadas de transmissão dentro da ISR.

5.5 Telemetria de Sensores

A função **sendTelemetryData()** opera exclusivamente no **modo Debug** e monta o **STATUS_BYTE** com as flags dos quatro sensores nos bits 0 a 3:

```
STATUS_BYTE = SENS1_flag | (SENS2_flag << 1) | (SENS3_flag << 2) | (SENS4_flag <<
```

3)

A transmissão só ocorre quando o `STATUS_BYTE` muda em relação ao valor anterior (`sensStatusByteLast`), evitando flood no barramento UART. O valor inicial de `sensStatusByteLast` é `0xFF`, forçando o envio na primeira iteração.

6. Modos de Operação

O firmware opera em dois modos seleccionáveis pelo comando `DEBUG_MODE_TOGGLE_MSG` (`0xDD`):

Modo	Valor	Descrição
<code>OP_MODE_FSM</code>	0	Modo normal: FSM completa em execução, telemetria desativada
<code>OP_MODE_DEBUG</code>	1	Modo debug: FSM suspensa, apenas comandos assíncronos e telemetria

A troca de modo é processada pela função `handleModeToggle()`, chamada no início de cada iteração do loop principal, com prioridade sobre as demais funções. Ao entrar no modo Debug, o motor é travado (`stepperStopEngaged()`), o flash é apagado e um padrão de pisca no LED de debug confirma a transição. Ao retornar ao modo FSM, o estado é redefinido para `STATE_IDLE` e `last_state` para `STATE_NONE`, forçando a execução das entry actions.

O comando `SW_RESET_MSG` (`0x33`) provoca um reset por software via `NVIC_SystemReset()`, com um delay de 50 ms para garantir que o frame de confirmação seja transmitido antes do reset.

7. Máquina de Estados Finitos (FSM)

A FSM principal da aplicação é implementada na função `FSM()`, chamada no loop principal somente quando `operationMode == OP_MODE_FSM`. O padrão utilizado separa explicitamente as **entry actions** (executadas uma única vez na transição de estado) das **actions de estado** (executadas a cada iteração).

A condição de entrada é verificada por `state != last_state`. Ao executar a entry action, `last_state` é igualado a `state`, fechando a condição.

7.1 Descrição dos Estados

STATE_IDLE (0): Estado inicial e de repouso. O flash é desligado, a cancela é fechada (`SetServoAngle(0)`) e o motor é habilitado em modo follow-steps. A cada iteração, é solicitado um movimento de 350 passos. A transição ocorre quando `SENS1_flag == 1` (objeto detectado na entrada da esteira), momento em que `objDetectedMSG_flag` é setada para notificar o Raspberry Pi.

STATE_OBJECT_DETECTED (1): O motor continua em movimento (350 passos por iteração). A transição ocorre quando `SENS2_flag == 1` (objeto sob a câmera), momento em que `classificationRequestMSG_flag` é setada, solicitando ao Raspberry Pi que realize a leitura do QR code e envie o destino.

STATE_WAIT_CLASSIFICATION (2): O flash é ligado para iluminar a cena de captura e o motor é parado com o eixo travado (`stepperStopEngaged()`). O firmware aguarda o recebimento de `ROUTE_A_RECEIVE_MSG`

(0xDA) ou `ROUTE_B_RECEIVE_MSG` (0xDB). Ao receber o destino, o frame é confirmado com `CMD_OK` e a flag de encaminhamento correspondente é setada.

STATE_ROUTE_A (3): A cancela é mantida fechada (`SetServoAngle(closeAngle)`), o flash é desligado e o motor retoma o movimento. A transição ocorre quando `SENS3_flag == 1` (objeto saiu pela Rota A), setando `routeASuccesDeliveryMSG_flag` e retornando a `STATE_IDLE`.

STATE_ROUTE_B (4): A cancela é aberta (`SetServoAngle(openAngle)`), o flash é desligado e o motor retoma o movimento. A transição ocorre quando `SENS4_flag == 1` (objeto saiu pela Rota B), setando `routeBSuccesDeliveryMSG_flag` e retornando a `STATE_IDLE`.

8. Procedimento de Inicialização e Handshake

Antes de entrar no loop principal, o firmware executa uma rotina de inicialização que bloqueia até que duas condições sejam satisfeitas:

- Todos os quatro sensores operacionais:** `SENS1_freq == 200 && SENS2_freq == 200 && SENS3_freq == 200 && SENS4_freq == 200`. Enquanto isso não ocorre, o LED de debug pisca rapidamente (100 ms).
- Handshake com o Raspberry Pi:** o STM32 aguarda receber `[0xAA][0x10]` (`SYS_RDY_MSG`). Após receber os sensores prontos + o handshake, responde com `[0x90][0x01]` (`SYS_INIT_MSG`) seguido de `[0x90][0x11]` (`MODE_FSM_MSG`), informando o modo de operação inicial. Enquanto aguarda o handshake com os sensores já prontos, o LED pisca lentamente (500 ms).

9. Controle Assíncrono (Modo Debug)

No modo Debug, a função `handleAsyncCommands()` processa os comandos recebidos para controle individual dos periféricos:

Comando	Ação
<code>LIGHT_EN</code> (0xE1)	Liga o pino <code>FLASH_PIN</code>
<code>LIGHT_DISABLE</code> (0xD1)	Desliga o pino <code>FLASH_PIN</code>
<code>GATE_OPEN</code> (0xE2)	<code>SetServoAngle(openAngle)</code>
<code>GATE_CLOSE</code> (0xD2)	<code>SetServoAngle(closeAngle)</code>
<code>STPR_EN</code> (0xE3)	<code>stepperFollowSteps()</code>
<code>STPR_DISABLE</code> (0xD3)	<code>stepperStopDisengaged()</code>
<code>STPR_FORWARD</code> (0xE4)	Define <code>stepperDirInst = 0</code>
<code>STPR_BACKWARD</code> (0xD4)	Define <code>stepperDirInst = 1</code>
<code>STPR_TGT_STPS</code> (0xE5)	<code>targetStepps = rxDATA</code>

Cada comando recebe uma confirmação `[0x90][CMD_eco]`. Comandos não reconhecidos retornam `[0x91]`.

10. Resumo das GPIOs

Pino	Direção	Função
DIR	Saída	Direção do motor de passo (A4988)
STEP	Saída	Pulsos de passo (A4988)
SLEEP	Saída	Enable/Sleep do driver A4988
dbgLED	Saída	LED de diagnóstico
FLASH_PIN	Saída	Acionamento do flash de iluminação